MODELARE ȘI SIMULARE

Săptămâna 10. Modele din date, AI + simulare și gemeni digitali

Suport de curs · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Obiective

1. să construiți și să interpretați un model de regresie (coeficienți, eroarea standard Sy, coeficientul de corelație r, R^2) și să distingeți performanța de antrenare de cea de test;
2. să recunoașteți supraînvățarea și să evaluați corect un model din date (validare, validare încrucișată, împărțire cronologică);
3. să construiți un model surogat al unei simulări scumpe, cu un plan Latin Hypercube și un proces gaussian, și să îl folosiți pentru calibrare;
4. să explicați ideea rețelelor neurale informate de fizică (PINN), a operatorilor neurali și a descoperirii ecuațiilor din date (SINDy);
5. să descrieți folosirea simulării pentru antrenarea sistemelor AI, decalajul sim-to-real și randomizarea domeniului;
6. să definiți un geamăn digital, să implementați asimilarea datelor cu un filtru Kalman (extins) și să folosiți geamănul pentru o decizie.

Cuvinte cheie: *model corelațional, regresie liniară, metoda celor mai mici pătrate, eroarea standard a estimării, coeficient de corelație, R^2, regresie multiplă, variabile indicator, gradient boosting, supraînvățare, generalizare, validare încrucișată, model de bază, model surogat, emulator, metamodel, Latin Hypercube, proces gaussian, kriging, cutie albă / gri / neagră, prognoză numerică a vremii, reanaliză ERA5, GraphCast, GenCast, AIFS, PINN, puncte de colocare, operator neural, simulare diferențiabilă, SINDy, regresie rară, învățare prin întărire, decalaj sim-to-real, randomizarea domeniului, geamăn digital, umbră digitală, asimilarea datelor, filtrul Kalman, câștig Kalman, stare augmentată, VVUQ, Destination Earth*

1. De la regresie la modele surogat

1.1. Modele mecaniste şi modele din date

În primele nouă săptămâni am construit modele **mecaniste**: am pornit de la mecanisme cunoscute – sosiri și serviri, legi de conservare, reguli locale, decizii ale agenților – iar parametrii (λ, μ, k, β) au avut un sens clar. Un astfel de model poate **extrapola** în situații nevăzute, dar cere înțelegerea sistemului și poate fi lent de simulat. **Modelele din date** pornesc doar de la observații; profesorul Holban (cap. 8) le numește **modele corelaționale**, pentru că descriu relațiile statistice dintre variabile fără să explice cauza. Ele sunt rapide după antrenare, dar funcționează bine doar în zona acoperită de date. Clasificarea în **cutie albă** (totul din mecanisme), **cutie neagră** (totul din date) și **cutie gri** (structura din fizică, o parte învățată) sugerează regula practică a săptămânii: folosim toată fizica pe care o cunoaștem sigur și învățăm din date doar ce lipsește.

Datele intră într-un studiu de simulare în mai multe roluri. Le-am folosit deja pentru **modelarea intrărilor** (S6), pentru **calibrare** (S7, S9) și pentru **validare** (S6). Săptămâna aceasta adaugă trei roluri noi: **modelele surogat**, învățate din rulări ale simulării; **componentele învățate** în interiorul unui model mecanist; și **asimilarea datelor**, în care senzorii corectează continuu starea modelului – baza gemenilor digitali.

1.2. Regresia liniară: exemplul Holban

Un model de regresie liniară simplă are forma $\hat{y} = a_0 + a_1 \cdot x$. Coeficienții se aleg astfel încât suma pătratelor erorilor $E(a_0, a_1) = \Sigma(y_i - a_0 - a_1 x_i)^2$ să fie minimă. Anulând derivatele parțiale obținem **ecuațiile normale**, cu soluția $a_1 = (n\Sigma xy - \Sigma x \Sigma y) / (n\Sigma x^2 - (\Sigma x)^2)$ și $a_0 = \bar{y} - a_1 \bar{x}$; dreapta trece mereu prin punctul mediilor $(\bar{x}, \bar{y})$. Pentru exemplul din curs ($x = 1, 3, 4, 6, 8, 9, 11, 14$; $y = 1, 2, 4, 4, 5, 7, 8, 9$) avem $n = 8$, $\Sigma x = 56$, $\Sigma y = 40$, $\Sigma xy = 364$, $\Sigma x^2 = 524$, de unde $a_1 = 672 / 1\,056 = 0{,}6364$ și $a_0 = 5 - 0{,}6364 \cdot 7 = 0{,}5455$ (exact 7/11 și 6/11).

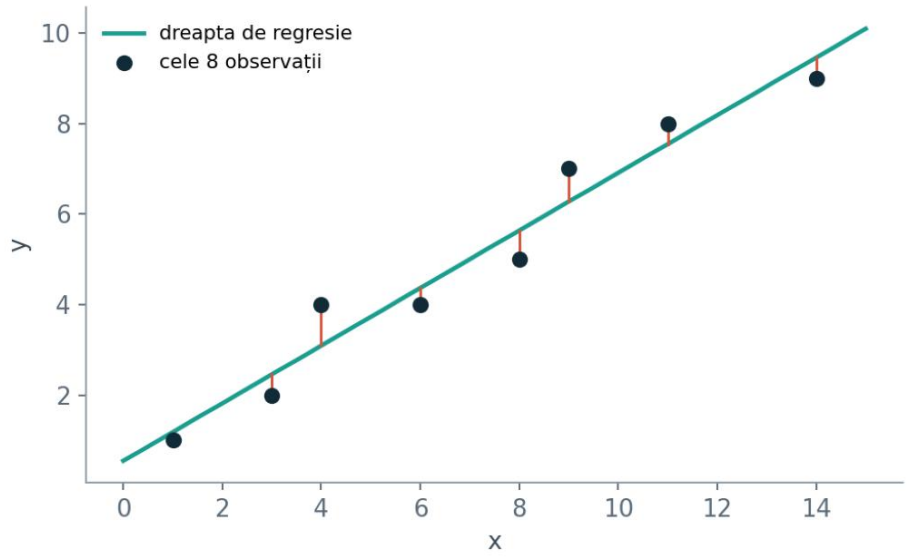


Figura 1. Exemplul Holban (cap. 8): cele 8 observații, dreapta de regresie și erorile (segmentele verticale)

Tabelul 1. Exemplul Holban: valorile calculate și pătratele erorilor

x	y	$\hat{y}$	$(y - \hat{y})^2$
1	1	1,18	0,0331
3	2	2,45	0,2066
4	4	3,09	0,8264
6	4	4,36	0,1322
8	5	5,64	0,4050
9	7	6,27	0,5289
11	8	7,55	0,2066
14	9	9,45	0,2066
Σ = 56	Σ = 40	Σ = 40	2,545

Calitatea modelului se judecă prin **eroarea standard a estimării** $S_y = \sqrt{(\Sigma(y_i - \hat{y}_i)^2/n)} = \sqrt{(2{,}545/8)} = 0{,}564$ și prin **coeficientul de corelație** $r = \sqrt{(1 - S_y^2/s_y^2)}$, unde $s_y^2 = \Sigma y^2/n - \bar{y}^2 = 256/8 - 25 = 7$ este dispersia lui y. Rezultă $r = 0{,}977$, egal cu coeficientul Pearson calculat direct, iar $R^2 = r^2 = 95{,}5\%$ este fracțiunea din variația lui y explicată de dreaptă. Refăcând calculul găsim două **neconcordanțe** în textul original: primul pătrat al erorii apare ca 0,324, corect fiind $(1 - 1{,}18)^2 = 0{,}0331$, iar coeficientul de corelație apare ca 0,99, deși din

datele cursului rezultă 0,974 cu Sy rotunjit la 0,6 și 0,977 cu valoarea exactă. Concluzia calitativă (relație puternic liniară) nu se schimbă, dar exemplul arată încă o dată de ce recalculăm numerele cu un program.

Regresia **multiplă** folosește mai multe variabile explicative. Exemplul din curs estimează performanța relativă a 209 procesoare: PRP = −55,9 + 0,0489·MYCT + 0,0153·MMIN + 0,0056·MMAX + 0,6410·CACH − 0,2700·CHMIN + 1,480·CHMAX (durata ciclului, memoria minimă și maximă, cache-ul, canalele de intrare–ieșire). Coeficienții se obțin matriceal, $a = (X^TX)^{-1}X^Ty$ (în practică np.linalg.lstsq), iar interpretarea cere atenție: când variabilele sunt corelate între ele, un coeficient poate avea un semn surprinzător, cum este cel negativ al lui CHMIN. „Liniar" înseamnă liniar în **coeficienți**: putem adăuga pătrate, logaritmi și **variabile indicator** (0/1) pentru categorii ca ora din zi. În sfârșit, o corelație bună permite predicția, dar nu spune ce se întâmplă dacă intervenim în sistem – **corelația nu este cauzalitate**.

1.3. Date reale: închirierile de biciclete din Londra

Setul de date din laboratorul 7 (London bike sharing) conține 17 414 de ore, între ianuarie 2015 și ianuarie 2017: numărul de biciclete închiriate, temperatura, umiditatea, vântul, codul vremii și calendarul. Antrenăm pe 2015 (8 643 de ore) și testăm pe 2016 (8 771 de ore) – o împărțire **cronologică**, care imită folosirea reală. Profilul orar are două vârfuri în zilele lucrătoare (3 864 la ora 8, 3 232 la ora 17) și o singură cocoașă în weekend, deci relația cu ora este puternic neliniară și depinde de tipul zilei. Corelația cu temperatura este doar 0,39.

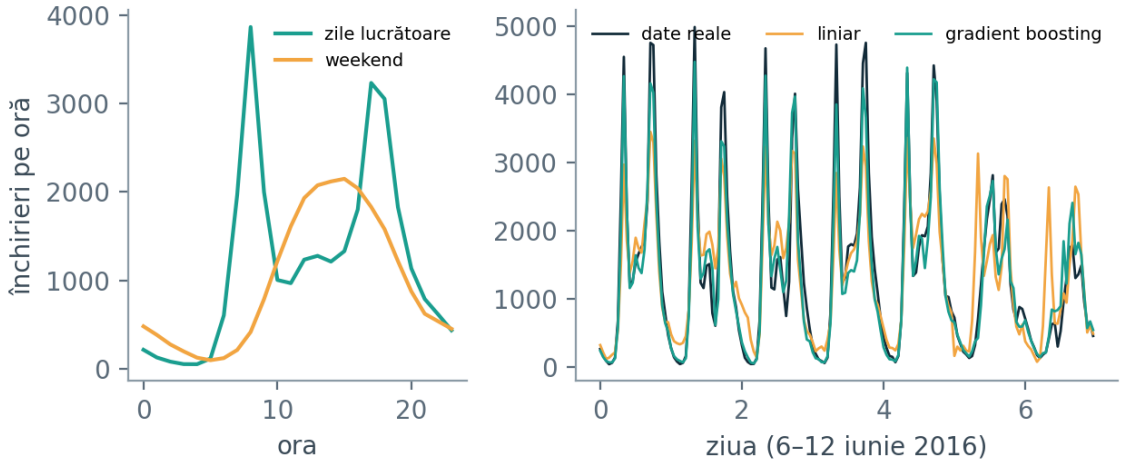


Figura 2. Stânga: numărul mediu de închirieri pe oră, în zilele lucrătoare și în weekend. Dreapta: săptămâna 6–12 iunie 2016 (nevăzută la antrenare) – datele reale, modelul liniar și gradient boosting

Tabelul 2. Trei modele pentru cererea orară de biciclete (antrenare 2015, test 2016)

Model	R^2 antrenare	R^2 test	RMSE test
Dreaptă: cnt = 128,2 + 79,1·t1	0,15	0,15	–
Liniar cu indicatori pentru oră și vreme	0,715	0,713	593
Gradient boosting (HistGradientBoostingRegressor)	0,970	0,934	284

Temperatura singură explică doar 15% din variație. Cu indicatori pentru fiecare oră și cu vremea, regresia liniară ajunge la $R^2 = 0,71$, dar presupune același profil orar în toate zilele și subestimează vârfurile. **Gradient boosting** construiește sute de arbori de decizie mici, fiecare corectând erorile celor de dinainte, și găsește

singur interacțiunile (de exemplu, că ora contează diferit în weekend): R^2 = 0,934 pe 2016, cu eroarea tipică înjumătățită. Performanța se raportează **pe datele de test**; scorul de antrenare (0,97) este aproape întotdeauna optimist.

1.4. Supraînvățare și generalizare

Pentru a vedea **supraînvățarea**, antrenăm polinoame de grad tot mai mare pe doar 20 de zile (cererea zilnică în funcție de temperatura medie). Eroarea de antrenare scade la fiecare creștere a gradului (de la 5 671 la 2 779), dar eroarea pe celelalte zile este minimă pentru gradele 1–2 (~6 612) și explodează apoi (642 869 la gradul 8, peste un milion de la gradul 10): polinomul a învățat zgomotul celor 20 de zile, nu relația reală. Evaluarea corectă folosește trei seturi – **antrenare** (coeficienții), **validare** (alegerea modelului) și **test** (raportul final, folosit o singură dată) –, **validarea încrucișată** cu k părți când avem puține date, împărțirea cronologică pentru serii de timp și **regularizarea** (ridge, lasso). Orice prelucrare (normalizare, selecție de variabile) se învață doar pe datele de antrenare, altfel apare scurgerea de informație. Este aceeași disciplină ca la validarea modelelor de simulare (S6): calibrarea joacă rolul antrenării, iar validarea pe date nefolosite rolul testului.

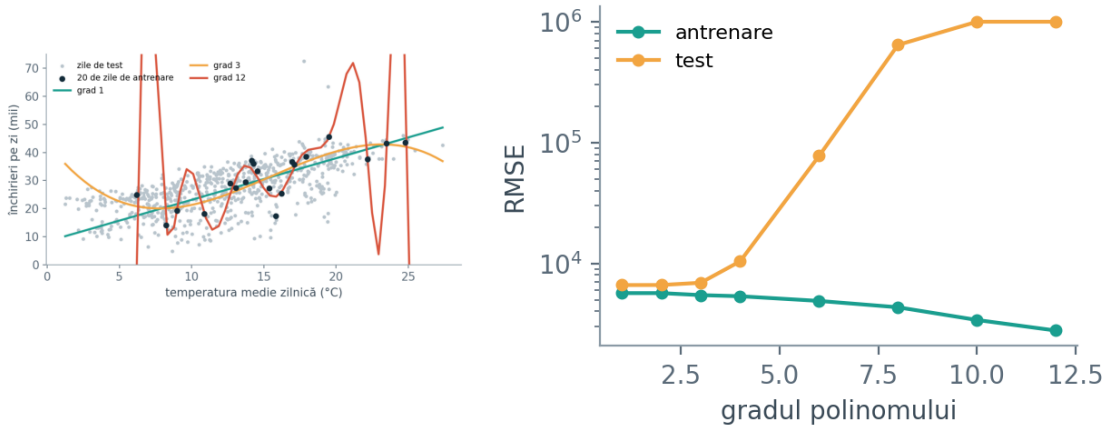


Figura 3. Stânga: polinoame de grad 1, 3 și 12 antrenate pe 20 de zile. Dreapta: eroarea de antrenare și de test în funcție de grad (scară logaritmică)

1.5. Modele surogat pentru simulări scumpe

Un **model surogat** (emulator, metamodel) aproximează funcția parametri ↦ rezultat a unui simulator. Se antrenează pe un număr mic de rulări alese cu grijă și apoi înlocuiește simulatorul acolo unde avem nevoie de mii de evaluări: optimizare, calibrare, analiza sensibilității și a incertitudinii, explorare interactivă. Pașii sunt: un **plan de experimente**, rulările simulatorului, antrenarea, validarea pe rulări noi și, dacă e nevoie, adăugarea de puncte acolo unde surogatul este cel mai nesigur. Planul **Latin Hypercube** (McKay, 1979) împarte fiecare parametru în n intervale și folosește fiecare interval o singură dată, acoperind uniform fiecare axă cu orice număr de rulări – spre deosebire de o grilă completă, al cărei cost crește exponențial cu numărul de parametri.

```
X = lhs(80)                          # rho in [0,1; 0,9], Cs2 in [0,2; 3]
y = [sim_queue(*x, seed=i) for i, x in enumerate(X)]   # Lindley, 20 000 clienti
gp = GaussianProcessRegressor(C() * RBF([.3, 1]) + WhiteKernel())
gp.fit(X, np.log(y))                 # invatam log W
mu, sd = gp.predict(X_nou, return_std=True)
```

Demo-ul săptămânii construiește surogatul unei cozi M/G/1 simulate cu recurența lui Lindley (S4), cu doi parametri: utilizarea ρ și variabilitatea servirii Cs^2. O simulare (20 000 de clienți) durează ~15 ms; avem 80 de rulări de antrenare și 300 de puncte de test, iar eroarea se măsoară față de formula exactă Pollaczek–Khinchine. Procesul gaussian are o eroare medie de 1,6%, mai mică decât a unei singure simulări (3,4%), pentru că netezește zgomotul rulărilor vecine, și este de ~9 569 de ori mai rapid. Procesul gaussian dă și o **incertitudine** pentru fiecare predicție (metoda se numește kriging în geostatistică); banda se lărgește unde nu avem rulări, ceea ce indică unde să rulăm în continuare. Alegerea **scării** contează: un polinom de grad 3 antrenat direct pe W are o eroare medie de 14,2%, iar pe log W de 2,2% – cunoașterea sistemului (W crește ca $1/(1-\rho)$) a contat mai mult decât un model complicat.

Tabelul 3. Patru surogate ale cozii M/G/1, antrenate pe aceleași 80 de simulări (erori relative pe 300 de puncte de test)

Surogat	Eroare medie	Eroare maximă	Timp / predicție
Regresie liniară	13,2%	41,7%	0,8 µs
Polinom de grad 3	2,2%	9,3%	1,6 µs
Rețea neurală (2 × 32)	5,5%	26,7%	1,1 µs
Proces gaussian	1,6%	11,6%	1,6 µs
O singură simulare (referință)	3,4%	–	15,3 ms

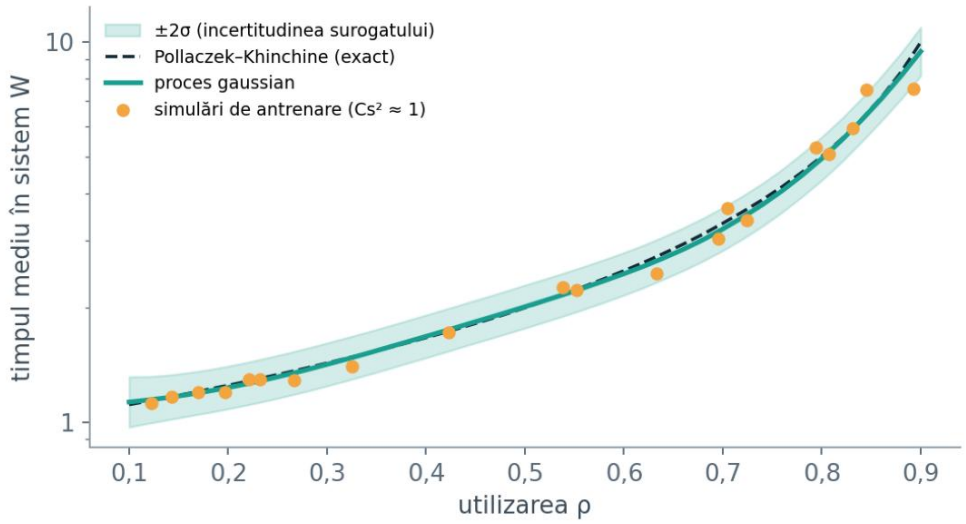


Figura 4. Surogatul cozii M/G/1 (secțiune la $Cs^2 = 1$): predicția procesului gaussian, banda ±2σ, formula exactă și simulările de antrenare

Aplicăm apoi aceeași idee modelului cu agenți din S9 (epidemia Holban, redusă la 2 500 de agenți). Cu doar 10 simulări de antrenare, emulatorul prezice vârful epidemiei în funcție de probabilitatea de transmisie p; 89% dintre 46 de simulări noi cad în banda ±2σ. Folosim emulatorul pentru calibrare: 100 000 de eșantioane a priori sunt evaluate în 0,05 s (cu simulatorul, ~4,8 ore), iar ponderarea după distanța față de vârful observat (1 190), ținând cont de zgomotul replicărilor și de incertitudinea emulatorului, dă $p = 0{,}098 \pm 0{,}012$ (adevărat: 0,1). Acesta este răspunsul la întrebarea din S9: dacă o simulare durează o oră, câteva zeci de rulări pentru emulator înlocuiesc sute de mii de ore de calcul.

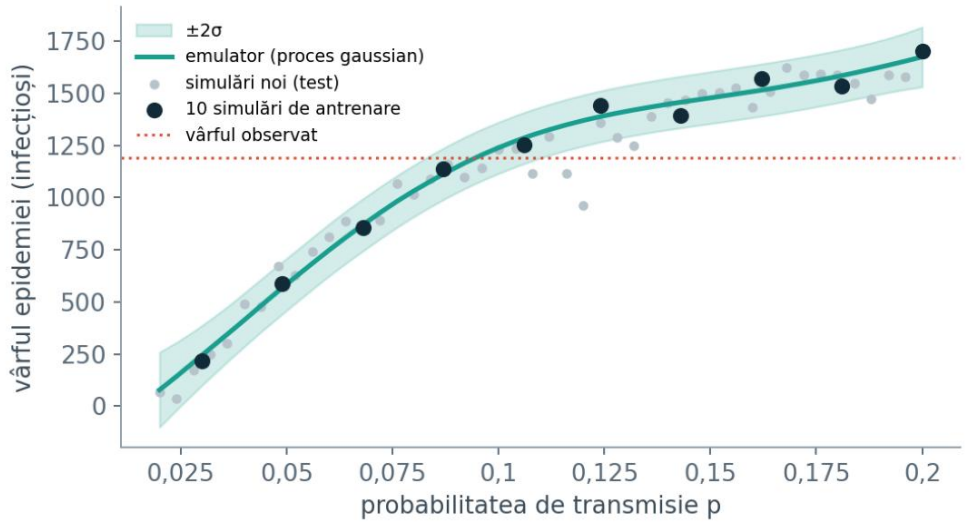


Figura 5. Emulatorul epidemiei din S9: 10 simulări de antrenare, predicția cu banda ±2σ, 46 de simulări noi de test și vârful observat folosit la calibrare

De reținut

Regresie: $a_1 = (n\Sigma xy - \Sigma x\Sigma y)/(n\Sigma x^2 - (\Sigma x)^2)$, $a_0 = \bar{y} - a_1\bar{x}$; $Sy = \sqrt{(\Sigma(y - \hat{y})^2/n)}$; $r = \sqrt{(1 - Sy^2/sy^2)}$; R^2 = fracțiunea de variație explicată. Exemplul Holban: $\hat{y} = 0{,}5455 + 0{,}6364x$, $r = 0{,}977$ (nu 0,99). Evaluăm numai pe date nefolosite; supraînvățarea: eroarea de antrenare scade, cea de test crește. Surogat = model rapid învățat din rulări (plan LHS + proces gaussian), cu incertitudine; util când avem nevoie de multe evaluări.

2. Fizică + învățare automată; simulare pentru AI

2.1. Prognoza meteo: cel mai mare surogat din lume

Prognoza numerică clasică (NWP) rezolvă ecuațiile atmosferei pe o grilă globală – la Centrul European (ECMWF), cu ~9 km rezoluție –, pornind de la o stare inițială obținută prin **asimilarea** a milioane de observații pe zi; incertitudinea vine din ansambluri de ~50 de rulări. Reanaliza **ERA5** reconstruiește starea atmosferei oră de oră din 1940 până azi și este setul de antrenare al modelelor AI. GraphCast (Google DeepMind, Science 2023) produce o prognoză pe 10 zile în sub un minut pe un TPU; **GenCast** (Nature, decembrie 2024) generează un ansamblu de 50+ prognoze pe 15 zile, este mai precis decât ansamblul fizic ENS pe 97,2% din ținte și rulează în ~8 minute pe un TPU v5; **NeuralGCM** (Nature 2024) este un model hibrid, cu nucleu dinamic diferențiabil și fizică învățată. ECMWF a introdus operațional modelul AIFS Single în februarie 2025 și ansamblul **AIFS ENS** la 1 iulie 2025, cu câștiguri de până la 20%, de peste 10 ori mai rapid și cu ~1 000 de ori mai puțină energie.

Modelele meteo învățate au și limite: învață din reanaliza produsă de modele fizice și pornesc din starea inițială obținută prin asimilare clasică; evenimentele extreme rare sunt slab reprezentate; rezoluția este de obicei mai mică (AIFS ~31 km față de 9 km), iar conservarea fizică nu este garantată. De aceea modelele fizice nu dispar – ECMWF rulează ambele tipuri în paralel. Sunt, de fapt, surogate la scară planetară: aceeași idee ca în demo-ul M/G/1, cu câștiguri de viteză de ordinul miilor.

2.2. Rețele neurale informate de fizică

O **PINN** (Raissi, Perdikaris și Karniadakis, 2019) este o rețea $T(t;\theta)$ antrenată pe date și pe ecuație în același timp. Funcția de cost conține eroarea față de măsurători plus reziduul ecuației diferențiale într-un număr mare de **puncte de colocare**, unde nu avem date; derivatele rețelei se calculează exact prin diferențiere automată, iar parametrii necunoscuți ai ecuației se învață împreună cu ponderile. Pentru răcirea lui Newton (S7), $dT/dt = -k\cdot(T - 20)$: $J(\theta, k) = (1/N)\Sigma(T(t_i) - T_i)^2 + \lambda\cdot(1/M)\Sigma(dT/dt(\tau_j) + k\cdot(T(\tau_j) - 20))^2$, cu $N = 6$ măsurători în primele 8 minute, $M = 41$ de puncte de colocare pe $[0; 40]$ minute și $\lambda = 100$.

```
def net(th, t):                      # 12 neuroni tanh
    z = np.tanh(np.outer(t / 40, W1) + b1)
    T = 20 + 70 * (z @ W2 + b2)
    dT = 70 * ((1 - z**2) * W1 / 40) @ W2       # derivata exacta
    return T, dT
cost = mean((T(t_obs) - T_obs)**2) + 100 * mean((dT(t_col) + k*(T(t_col) - 20))**2)
```

Rezultatul: rețeaua antrenată doar pe date trece prin cele 6 puncte, dar după minutul 8 deviază (eroare maximă 3,0 °C, RMSE 1,89 °C pe 0–40 min); PINN urmează soluția exactă pe tot intervalul (eroare maximă 0,07 °C, RMSE 0,05 °C) și estimează $k = 0{,}0803$ (adevărat 0,08). Rețeaua are 38 de parametri și doar 6 măsurători, deci fără ecuație problema ar fi subdeterminată: ecuația acționează ca o regularizare. PINN ajută mai ales la **probleme inverse**, cu date rare și fizică incompletă; pentru o problemă directă bine pusă, solverele clasice (RK45, elemente finite) rămân de obicei mai rapide și mai precise, iar antrenarea unei PINN este delicată (ponderea λ, sisteme rigide, fronturi abrupte).

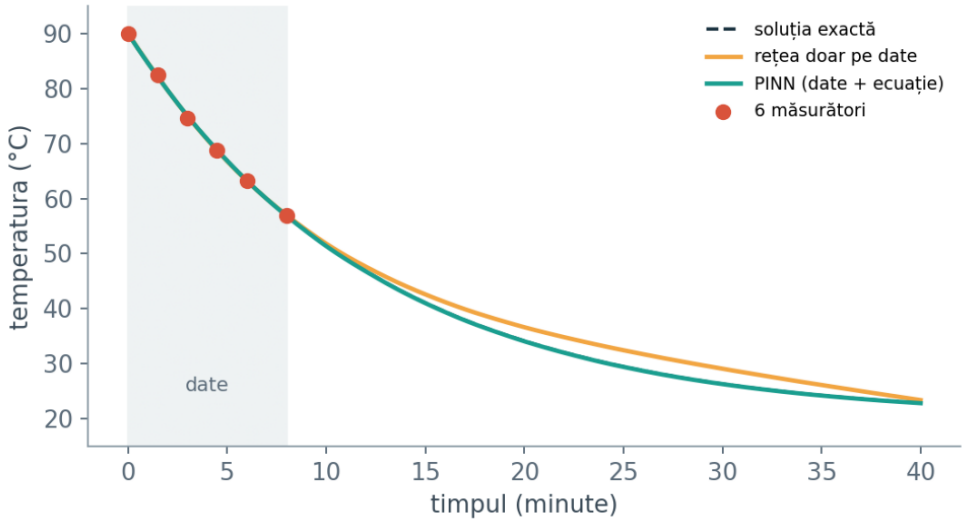


Figura 6. Răcirea cafelei din 6 măsurători (zona gri): rețeaua antrenată doar pe date, PINN și soluția exactă

Un **operator neural** (FNO – Li et al., ICLR 2021; DeepONet – Lu et al., 2021) învață aplicația care duce o funcție de intrare (condiția inițială, coeficienții) în soluție, pentru o clasă întreagă de probleme; se antrenează pe mii de rulări ale unui solver clasic și rezolvă apoi probleme noi în milisecunde, independent de rezoluție – un surogat ale cărui intrări și ieșiri sunt câmpuri. **Simularea diferențiabilă** scrie simulatorul într-un cadru cu diferențiere automată (JAX, PyTorch, Warp, MuJoCo MJX), astfel încât gradientul rezultatului în raport cu parametrii se obține prin toți pașii simulării; calibrarea și controlul devin coborâre pe gradient. Limita: evenimentele discrete (S5), coliziunile și haosul (S7) fac gradientul inexistent sau instabil.

2.3. Descoperirea ecuațiilor din date: SINDy

SINDy (Brunton, Proctor și Kutz, PNAS 2016) scrie derivatele ca o combinație a termenilor dintr-o bibliotecă: $dX/dt \approx \Theta(X)\cdot\Xi$, unde Θ conține coloanele 1, x, y, x^2, xy, y^2, iar Ξ trebuie să fie **rar** (cât mai mulți coeficienți nuli). Algoritmul STLSQ rezolvă prin cele mai mici pătrate, anulează coeficienții sub un prag (aici 0,05) și reajustează restul, repetat. Pe datele Lotka–Volterra ($\dot{x} = 1{,}1x - 0{,}4xy$, $\dot{y} = -0{,}4y + 0{,}1xy$), metoda regăsește exact cei 4 termeni și anulează ceilalți 8, inclusiv cu zgomot.

Tabelul 4. Coeficienții găsiți de SINDy pe datele Lotka–Volterra (ceilalți 8 coeficienți ies zero în toate cazurile)

Date	ẋ: x	ẋ: xy	ẏ: y	ẏ: xy
Modelul adevărat	1,1	−0,4	−0,4	0,1
Fără zgomot	1,099	-0,400	-0,400	0,100
Zgomot 1%	1,100	-0,400	-0,399	0,100
Zgomot 5% (netezit Savitzky–Golay)	1,078	-0,388	-0,396	0,098

Rezultatul este o **formulă** citibilă, care se poate interpreta și extrapola – o cutie albă obținută din date. Limitele: un termen care lipsește din bibliotecă nu poate fi găsit (biblioteca exprimă ce credem despre sistem), coordonatele contează, iar derivatele din date zgomotoase sunt dificile (netezim înainte de derivare). Regresia simbolică (PySR, AI Feynman) caută și forma termenilor. Kepler a făcut același lucru de mână, găsind legile mișcării planetelor din măsurătorile lui Tycho Brahe.

2.4. Simularea care antrenează AI

Roboții, dronele și mașinile autonome se antrenează mai ales în simulare: este **sigur**, **rapid** (mii de copii în paralel pe un GPU), permite generarea la cerere a **evenimentelor rare**, oferă **etichete perfecte** și **reproductibilitate**. Învățarea prin întărire are nevoie de milioane de episoade, practic imposibile pe sisteme reale. Unelte tipice: MuJoCo (open source din 2022), Isaac Sim / Isaac Lab, interfața Gymnasium, CARLA pentru conducere autonomă, motoare de jocuri și, mai nou, **modele ale lumii** – rețele generative care învață să simuleze din video. Problema centrală este **decalajul sim-to-real**: o politică optimizată exploatează toate detaliile simulatorului, inclusiv pe cele greșite (parametri, întârzieri, saturări, senzori), iar pe sistemul real performanța poate scădea dramatic. Remediile sunt identificarea sistemului (calibrarea simulatorului), **randomizarea domeniului** (Tobin et al., 2017: antrenăm pe o familie de simulatoare cu parametri aleatori, ca realitatea să pară încă o variantă) și ajustarea pe puține date reale.

```
def cost(Kp, Kd, m=1.0, b=0.5, delay=0, dt=0.02):
    x = v = 0.0; buf = [0.0] * (delay + 1); J = 0.0
    for k in range(500):                          # 10 s
        u = np.clip(Kp*(1 - x) - Kd*v, -20, 20)
        buf.append(u); ua = buf.pop(0)            # intarzierea actuatorului
        v += dt * (ua / m - b * v); x += dt * v
        J += dt * (x - 1)**2
    return J
```

Demo-ul reglează un controler PD care duce o masă în poziția 1. Simulatorul nominal are m = 1, b = 0,5 și nicio întârziere; sistemul „real” are m = 1,6, b = 0,3 și o întârziere a motorului de 0,12 s. Reglat pe simulatorul nominal, controlerul ales (Kp = 186, Kd = 24) are costul 0,173 în simulare, dar 11,56 pe sistemul real, unde oscilează; reglat pe 12 medii randomizate ($m \in [0{,}6; 2]$, $b \in [0{,}2; 0{,}8]$, întârziere 0–0,16 s), controlerul (Kp =

18, Kd = 6) costă 0,238 în simulare și 0,384 pe sistemul real. Pe 200 de medii noi, controlerul nominal eșuează în 55% din cazuri, cel robust în 9,5%. Optimizarea pe un singur model este o formă de supraînvățare – pe simulator, nu pe date.

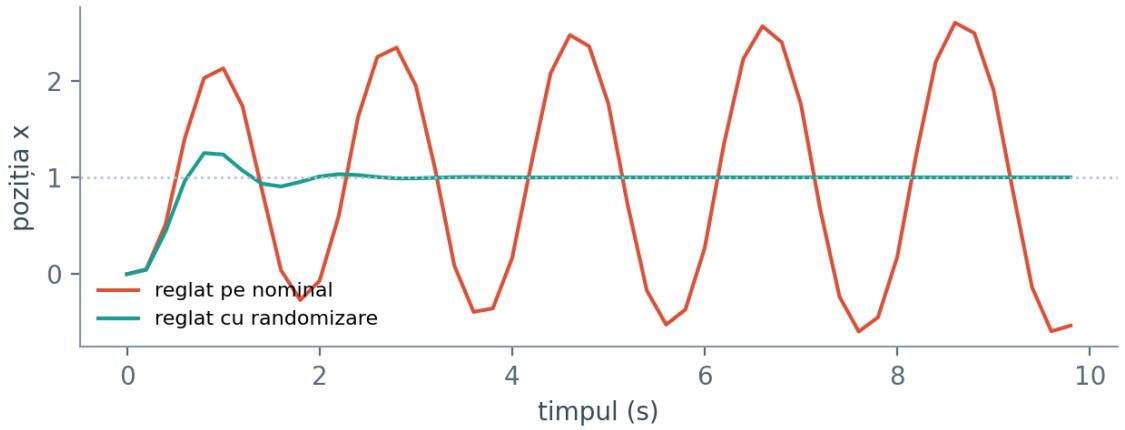


Figura 7. Pe sistemul real (m = 1,6, b = 0,3, întârziere 0,12 s): controlerul reglat pe simulatorul nominal oscilează, cel reglat cu randomizare se stabilizează

În robotică, OpenAI (2019) a antrenat numai în simulare, cu randomizare automată, o mână care rezolvă cubul Rubik și rezistă unor perturbări nevăzute; roboții patrupezi învață să meargă în câteva minute cu mii de copii simulate în paralel (ETH Zürich, Isaac Gym, 2021). Waymo raportează aproape 200 de milioane de mile autonome pe drumuri publice și miliarde de mile în simulare, unde incidentele reale sunt reluate în mii de variante. Riscurile – exploatarea erorilor simulatorului, recompense prost specificate, „prăbușirea" modelelor antrenate repetat pe date generate (Shumailov et al., Nature 2024), încrederea falsă – se reduc la întrebarea din S6: cât de valid este modelul pentru scopul în care îl folosim?

De reținut

Modelele meteo AI (GraphCast, GenCast, AIFS) sunt surogate rapide, construite pe date produse de fizică. PINN: ecuația intră în funcția de cost; bună pentru probleme inverse și date rare. Operatori neurali: surogate pentru câmpuri. SINDy: regresie rară pe o bibliotecă de termeni – găsește formula. Simularea antrenează AI; decalajul sim-to-real se reduce prin calibrare și randomizarea domeniului.

3. Gemeni digitali și sinteza cursului

3.1. Ce este un geamăn digital

Raportul National Academies (2024) definește geamănul digital ca un set de construcții informaționale virtuale care imită structura, contextul și comportamentul unui sistem natural, tehnic sau social, este **actualizat dinamic** cu date de la geamănul fizic, are **capacitate predictivă** și **informează decizii** care produc valoare; interacțiunea **bidirecțională** dintre virtual și fizic este esențială. Kritzinger et al. (2018) disting **modelul digital** (date și decizii manuale – toate modelele din S1–S9), **umbra digitală** (date automate, decizii manuale) și **geamănul digital** (ambele sensuri automate sau asistate). Componentele – modelul, datele, asimilarea, predicția cu incertitudine, decizia și VVUQ continuu – sunt metode deja cunoscute, cu o singură piesă nouă: **asimilarea datelor**.

3.2. Asimilarea datelor: filtrul Kalman

Modelul singur este neted, dar deviază; senzorul singur urmărește realitatea, dar are zgomot și măsoară doar o parte din stare. **Filtrul Kalman** (1960) le combină în doi pași. **Predicția**: $\hat{x}^- = f(\hat{x})$, $P^- = F \cdot P \cdot F^T + Q$ – avansăm starea cu modelul și creștem incertitudinea cu zgomotul de proces Q. **Corecția**: $K = P^- H^T (H P^- H^T + R)^{-1}$, $\hat{x} = \hat{x}^- + K \cdot (y - H\hat{x}^-)$, $P = (I - KH) \cdot P^-$ – acceptăm fracțiunea K din **inovația** $y - H\hat{x}^-$. Într-o dimensiune, $K = P^-/(P^- + R)$ și estimarea este o medie ponderată: modelul prezice 35 °C cu dispersia 1, senzorul arată 37 °C cu dispersia 0,25, deci K = 0,8, $\hat{x}$ = 36,6 °C, iar dispersia finală 0,2 este mai mică decât a oricărei surse. Pentru modele neliniare folosim filtrul **extins** (EKF, liniarizare cu jacobianul F); un parametru necunoscut devine componentă a stării (**stare augmentată**); în meteorologie se folosesc ansambluri (EnKF), iar pentru modele cu agenți filtre cu particule.

3.3. Demo: geamănul unei camere de servere

Modelul este un bilanț termic (S7): $C \cdot dT/dt = P - k \cdot (T - T_a)$, cu C = 50, T_a = 18 °C, P = 60 și eficiența răcirii k = 4 (echilibru 33 °C). După minutul 180, k scade liniar până la 2,4 la minutul 240 (echilibru 43 °C, peste limita de 40 °C). Senzorul măsoară temperatura o dată pe minut, cu σ = 0,5 °C. Geamănul folosește un EKF cu starea augmentată [T, k]: k nu se măsoară, dar este lăsat să varieze puțin (zgomot de proces), iar prin covarianța cu T fiecare surpriză în temperatură îl corectează.

```
x = np.array([33.0, 4.0]); P = np.diag([1.0, 0.5])      # stare [T, k]
Q = np.diag([0.05**2, 0.004]); R = 0.5**2; H = np.array([1.0, 0.0])
for t in range(360):
    T, k = x
    x = np.array([T + (Pw - k*(T - Ta)) / C, k])            # predictie
    F = np.array([[1 - k/C, -(T - Ta)/C], [0, 1]]); P = F @ P @ F.T + Q
    K = P @ H / (H @ P @ H + R)                              # corectie
    x = x + K * (y[t] - x[0]); P = P - np.outer(K, H) @ P
```

Rezultate: modelul fără date (k = 4 fix) greșește cu 7,92 °C după degradare, senzorul are o eroare tipică de 0,48 °C, iar geamănul de 0,20 °C – mai bun decât ambele. Eficiența estimată urmărește degradarea; alarma (k estimat sub 3,4, adică 85% din nominal) apare la minutul 211, cu 31 de minute după începutul degradării și înainte ca temperatura să depășească limita. La final, k = 2,58 ± 0,15 (adevărat 2,4). La minutul 240 (T = 39,1 °C) comparăm două scenarii: cu sarcina completă temperatura depășește 40 °C în 10 minute (echilibru 41,3 °C), iar cu sarcina redusă cu 20% rămâne sub limită (echilibru 36,6 °C). Decizia închide bucla: sistemul fizic se schimbă, geamănul verifică dacă predicția s-a adeverit, iar incertitudinea (k poate scădea în continuare) trebuie comunicată.

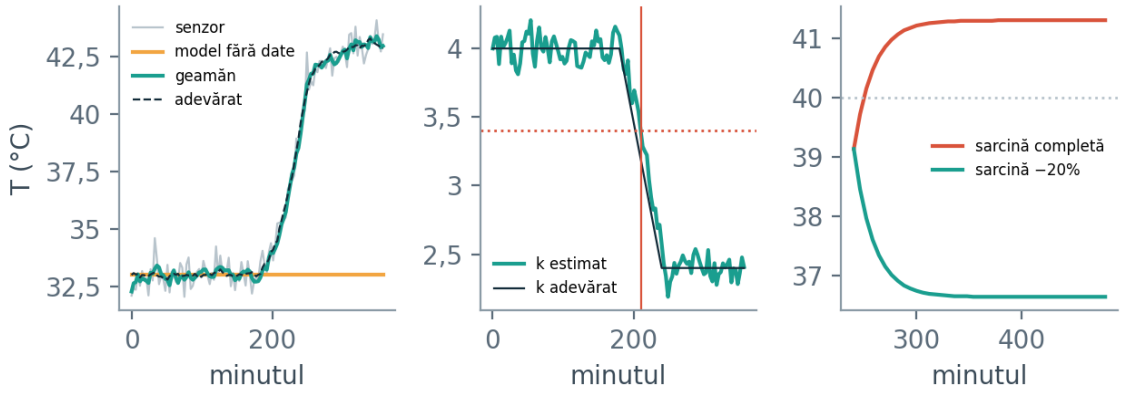


Figura 8. Geamănul camerei de servere. Stânga: temperatura (senzor, model fără date, geamăn, valoarea adevărată). Mijloc: eficiența răcirii estimată și pragul de alarmă. Dreapta: scenariile de la minutul 240

3.4. Gemeni digitali în practică

Gemenii digitali sunt folosiți unde sistemul este scump sau critic, există date continue și decizii repetate: linii de producție, motoare de avion și turbine (mentenanță la nevoie), rețele electrice, modele ale inimii unui pacient, orașe (trafic, inundații), centre de date. **Destination Earth**, inițiativa Comisiei Europene cu ECMWF, ESA și EUMETSAT, a lansat în iunie 2024 primele sisteme: geamănul extremelor meteo și geamănul adaptării la schimbările climatice, care rulează pe supercalculatoarele EuroHPC. Provocările sunt validarea care „expiră" când sistemul se schimbă (VVUQ continuu), propagarea incertitudinii până la decizie, calitatea datelor, securitatea și costul. Un geamăn nu merită pentru o decizie unică, fără senzori relevanți, fără decizii de luat sau când un simplu prag pe un senzor rezolvă problema – recomandarea este să începem mic.

3.5. Sinteza cursului

Tabelul 5. Cele 10 săptămâni ale cursului

S	Tema	Ideea de reținut
1	Sisteme, modele, simulare	ciclul de modelare: întrebare, model, verificare, validare, experimente
2	Aleatorul și metoda Monte Carlo	generatori, transformarea inversă; eroarea scade ca $1/\sqrt{n}$
3	Procese stocastice, lanțuri Markov	starea de echilibru $\pi P = \pi$; plimbări aleatoare
4	Teoria cozilor	$L = \lambda W$; M/M/1, Pollaczek–Khinchine; variabilitatea costă
5	Simularea cu evenimente discrete	lista de evenimente; ceasul sare între evenimente; SimPy
6	Date de intrare, rezultate, validare	replicări, intervale de încredere; verificare ≠ validare
7	Sisteme dinamice continue	EDO, Euler / RK4, stabilitate, rigiditate; SIR, Lotka–Volterra
8	Automate celulare, particule, HPC	reguli locale, comportament global; Verlet; Amdahl
9	Agenți și sisteme complexe	emergență, rețele, ODD, calibrare ABC
10	Date, AI + simulare, gemeni digitali	surogate, PINN, sim-to-real, filtrul Kalman

Alegerea metodei pornește de la întrebarea ce se schimbă în sistem și cum: mărimi continue – ecuații diferențiale (S7); evenimente discrete și cozi – formule (S4) sau simulare cu evenimente discrete (S5); interacțiuni locale în spațiu – automate celulare și particule (S8); indivizi diferiți care decid – agenți (S9). Peste orice ramură adăugăm Monte Carlo și replicări când intrările sunt nesigure (S2, S6), surogate când simularea este lentă și gemeni digitali când avem date în timp real și decizii repetate (S10). Principiile care s-au repetat: întâi întrebarea, apoi cel mai simplu model care răspunde la ea; o singură rulare nu înseamnă nimic; verificare și validare pentru scopul declarat; recalcularea numerelor din surse (am găsit erori în aproape fiecare material); reproductibilitate. „Toate modelele sunt greșite, dar unele sunt utile" (G. E. P. Box).

De reținut

Geamăn digital = model + date + asimilare + predicție + decizie, într-o buclă bidirecțională (National Academies, 2024); model digital → umbră digitală → geamăn. Kalman: predicție ($\hat{x}^- = f(\hat{x})$, $P^- = FPF^T + Q$), corecție ($K = P^-H^T(HP^-H^T + R)^{-1}$); în 1D, $K = P^-/(P^- + R)$ – medie ponderată. Stare augmentată:

estimăm parametri nemăsurați. Demo: RMSE model 7,92 / senzor 0,48 / geamăn 0,20 °C; alarma la minutul 211. VVUQ continuu.

Glosar RO – EN

Română	English
model corelațional	correlational model
regresie liniară	linear regression
metoda celor mai mici pătrate	least squares
eroarea standard a estimării	standard error of the estimate
coeficient de corelație	correlation coefficient
coeficient de determinare (R^2)	coefficient of determination
variabilă indicator	indicator (dummy) variable
supraînvățare	overfitting
generalizare	generalisation
validare încrucișată	cross-validation
scurgere de informație	data leakage
model surogat	surrogate model
emulator	emulator
plan de experimente	design of experiments
eșantionare Latin Hypercube	Latin hypercube sampling
proces gaussian	Gaussian process
cutie gri	grey box
reanaliză	reanalysis
rețea neurală informată de fizică	physics-informed neural network (PINN)
puncte de colocare	collocation points
operator neural	neural operator
simulare diferențiabilă	differentiable simulation
regresie rară	sparse regression
învățare prin întărire	reinforcement learning
decalaj sim-to-real	sim-to-real gap
randomizarea domeniului	domain randomisation
geamăn digital	digital twin

Română	English
umbră digitală	digital shadow
asimilarea datelor	data assimilation
filtrul Kalman (extins)	(extended) Kalman filter
câștig Kalman	Kalman gain
inovație	innovation
mentenanță predictivă	predictive maintenance

Întrebări de autoevaluare

1. Un model de regresie are **$R^2 = 0{,}95$** pe datele folosite la construcție. Ce putem afirma cu siguranță?

a) x este cauza lui y

b) modelul va avea $R^2 \approx 0{,}95$ și pe date noi

c) 95% din variația lui y din aceste date este explicată de model

d) 95% dintre puncte sunt exact pe dreaptă

2. Prin ce punct trece întotdeauna dreapta de regresie obținută prin metoda celor mai mici pătrate?

a) prin punctul mediilor $(\bar{x}, \bar{y})$

b) prin origine (0, 0)

c) prin primul și ultimul punct al datelor

d) prin punctul cu cea mai mare eroare

3. Care este semnul tipic al **supraînvățării** când creștem complexitatea modelului?

a) ambele erori, de antrenare și de test, cresc

b) eroarea de antrenare scade, iar eroarea de test crește

c) eroarea de antrenare crește, iar eroarea de test scade

d) ambele erori scad la zero

4. Cum împărțim datele orare de închirieri de biciclete în antrenare și test pentru un model care va prezice cererea de mâine?

a) aleator, 80% / 20%

b) antrenăm pe toate datele și testăm tot pe ele

c) alegem pentru test orele cu cele mai mari valori

d) cronologic: antrenăm pe trecut (2015), testăm pe viitor (2016)

5. Ce este un **model surogat** (emulator) al unei simulări?

a) un model rapid, învățat din câteva rulări ale simulării, care îi aproximează rezultatele

b) o versiune a simulării cu mai puține replicări

c) simularea rulată pe un calculator mai rapid

d) un model mecanist mai detaliat decât simularea

6. Ce proprietate are un plan **Latin Hypercube** cu n puncte?

a) punctele formează o grilă completă

b) toate punctele sunt în centrul domeniului

c) fiecare parametru este împărțit în n intervale, iar fiecare interval este folosit exact o dată

d) numărul de puncte crește exponențial cu numărul de parametri

7. Ce avantaj principal are un **proces gaussian** ca surogat, față de o regresie polinomială?

a) este întotdeauna mai rapid la predicție

b) dă, pe lângă predicție, și o incertitudine (σ) pentru fiecare punct

c) nu are nevoie de date de antrenare

d) funcționează doar pentru un singur parametru

8. În demo-ul M/G/1, de ce am antrenat surogatul pe **log W** în loc de W?

a) pentru că W este întotdeauna negativ

b) pentru că logaritmul face simularea mai rapidă

c) pentru că procesul gaussian nu acceptă valori mai mari ca 1

d) pentru că W crește exploziv când $\rho \to 1$, iar pe scara log erorile relative au aceeași greutate peste tot

9. De ce modelele meteo AI (GraphCast, GenCast, AIFS) **nu pot înlocui complet** astăzi modelele fizice?

a) datele de antrenare (reanaliza ERA5) și starea inițială provin din modele fizice cu asimilare

b) sunt mai lente decât modelele fizice

c) nu pot produce ansambluri

d) nu au fost folosite niciodată operațional

10. Ce conține funcția de cost a unei **PINN**?

a) doar eroarea față de măsurători

b) doar reziduul ecuației diferențiale

c) eroarea față de măsurători plus reziduul ecuației în puncte de colocare

d) numărul de parametri ai rețelei

11. Rulăm SINDy pe datele Lotka–Volterra, dar biblioteca are doar termenii 1, x, y, x^2, y^2 (fără **xy**). Ce obținem?

a) exact modelul adevărat, cu alți coeficienți

b) un model aproximativ, cu mai mulți termeni, care nu este cel adevărat

c) SINDy adaugă automat termenul lipsă

d) o eroare: algoritmul nu converge

12. Ce înseamnă **randomizarea domeniului** la antrenarea în simulare?

a) alegem aleator ordinea episoadelor

b) adăugăm zgomot în recompensă

c) antrenăm direct pe sistemul real, în condiții aleatoare

d) antrenăm pe o familie de simulatoare cu parametri aleși aleator, ca realitatea să pară încă o variantă

13. În demo, controlerul reglat pe simulatorul nominal costă 0,173 în simulare și 11,56 pe sistemul real. Cauza principală?

a) a exploatat un simulator fără întârziere; câștigurile mari devin instabile cu întârziere

b) zgomotul numeric al metodei Euler

c) grila de căutare a fost prea rară

d) costul a fost calculat greșit

14. Care dintre următoarele este un **geamăn digital** în sensul definiției National Academies (2024)?

a) un model 3D detaliat al unei fabrici, folosit la prezentări

b) o simulare calibrată o singură dată, anul trecut

c) un model actualizat automat din senzori, care prezice defecțiunile și programează mentenanța

d) un tablou de bord cu valorile senzorilor în timp real

15. Înlocuim senzorul unui filtru Kalman cu unul mult mai zgomotos (R mai mare). Ce se întâmplă?

a) câștigul K crește: credem mai mult senzorul

b) câștigul K scade: credem mai mult modelul; estimarea este mai netedă, dar reacționează mai lent

c) nimic: filtrul elimină orice zgomot

d) filtrul nu mai poate estima parametrii nemăsurați

16. Trei observații: (1; 2), (2; 4), (3; 5). Calculați panta a_1 a dreptei de regresie prin metoda celor mai mici pătrate.

(Răspuns numeric)

17. Pentru o regresie, eroarea standard a estimării este Sy = 0,5, iar dispersia lui y este $s_y^2 = 4$. Cât este $R^2 = 1 - S_y^2/s_y^2$? (4 zecimale)

(Răspuns numeric)

18. Filtru Kalman într-o dimensiune: predicția este 20 °C cu dispersia $P^- = 4$, iar senzorul arată 22 °C cu dispersia $R = 1$. Care este estimarea corectată (°C)?

(Răspuns numeric)

19. O simulare are 4 parametri. Câte rulări cere o grilă completă cu 5 valori pentru fiecare parametru? (Un plan Latin Hypercube poate folosi oricâte, de exemplu 40.)

(Răspuns numeric)

20. Geamănul camerei de servere: $C \cdot dT/dt = P - k \cdot (T - T_a)$, cu $T_a = 18$ °C și $P = 60$. Ce temperatură de echilibru (°C) rezultă după ce eficiența răcirii scade la $k = 2{,}4$?

(Răspuns numeric)

Răspunsuri

1. c) – R^2 este fracțiunea din variația lui y explicată de model, pe datele pe care a fost calculat; nu spune nimic despre cauzalitate sau despre date noi.

2. a) – Din ecuația normală pentru a_0 rezultă $a_0 = \bar{y} - a_1\bar{x}$, deci $\hat{y}(\bar{x}) = \bar{y}$: dreapta trece prin centrul de greutate al datelor.

3. b) – Un model mai flexibil se potrivește tot mai bine datelor de antrenare (inclusiv zgomotului), dar prezice tot mai prost date noi – ca polinomul de grad 12 pe 20 de zile.

4. d) – Pentru serii de timp, împărțirea aleatoare pune în antrenare ore vecine cu cele de test și dă o estimare optimistă; împărțirea cronologică imită folosirea reală.

5. a) – Surogatul aproximează funcția parametri ↦ rezultat a simulatorului și înlocuiește simulatorul acolo unde avem nevoie de multe evaluări.

6. c) – LHS (McKay, 1979) acoperă uniform fiecare axă cu orice număr de puncte, spre deosebire de grila completă, al cărei cost crește exponențial.

7. b) – Banda ±2σ se lărgește unde nu avem rulări; incertitudinea indică unde să rulăm simulatorul în continuare (optimizare bayesiană, eșantionare adaptivă).

8. d) – Pe scara directă, punctele cu W mare domină costul; același polinom de grad 3 are 14,2% eroare medie pe W și 2,2% pe log W.

9. a) – Modelele AI sunt mult mai rapide, pot produce ansambluri și sunt operaționale la ECMWF din 2025, dar se sprijină pe date și stări inițiale produse de fizică.

10. c) – J = eroarea pe date + $\lambda \cdot$ reziduul ecuației; ecuația regularizează rețeaua acolo unde nu avem date și permite estimarea parametrilor necunoscuți (k).

11. b) – SINDy poate alege doar termeni din bibliotecă; biblioteca exprimă cunoașterea noastră despre sistem.

12. d) – Tobin et al. (2017): o politică robustă pe toată familia de parametri (masă, frecare, întârzieri) se transferă mai bine pe sistemul real.

13. a) – Este decalajul sim-to-real: optimizarea a exploatat exact detaliul în care simulatorul era greșit (lipsa întârzierii de 0,12 s).

14. c) – Doar varianta C are actualizare dinamică, predicție și decizii care se întorc asupra sistemului; B este un model digital, D o umbră digitală.

15. b) – În 1D, $K = P^{-}/(P^{-} + R)$: un R mai mare scade K, deci măsurătorile contează mai puțin, iar schimbările reale sunt detectate mai târziu.

16. 1,5 – $n = 3$, $\Sigma x = 6$, $\Sigma y = 11$, $\Sigma xy = 25$, $\Sigma x^2 = 14$; $a_1 = (3 \cdot 25 - 6 \cdot 11) / (3 \cdot 14 - 36) = 9 / 6 = 1{,}5$.

17. 0,9375 – $R^2 = 1 - 0{,}25 / 4 = 1 - 0{,}0625 = 0{,}9375$; $r = \sqrt{0{,}9375} \approx 0{,}968$.

18. 21,6 – $K = P^{-} / (P^{-} + R) = 4 / 5 = 0{,}8$; $\hat{x} = 20 + 0{,}8 \cdot (22 - 20) = 21{,}6$ °C, cu dispersia $(1 - 0{,}8) \cdot 4 = 0{,}8$.

19. 625 – $5^4 = 625$ de rulări; costul grilei crește exponențial cu numărul de parametri.

20. 43 – La echilibru $dT/dt = 0$, deci $T = T_a + P/k = 18 + 60 / 2{,}4 = 18 + 25 = 43$ °C (față de 33 °C cu k = 4).

Bibliografie

- Ș. Holban – Tehnici de modelare și simulare, cap. 8: modele corelaționale (materiale de curs); setul de date London bike sharing (laboratorul 7).
- A. M. Law – Simulation Modeling and Analysis, ed. 6, McGraw-Hill, 2024 (metamodele și planuri de experimente).
- J. P. C. Kleijnen – Design and Analysis of Simulation Experiments, ed. 2, Springer, 2015.
- C. E. Rasmussen, C. K. I. Williams – Gaussian Processes for Machine Learning, MIT Press, 2006 (gaussianprocess.org/gpml).
- M. D. McKay, R. J. Beckman, W. J. Conover – A comparison of three methods for selecting values of input variables in the analysis of output from a computer code, Technometrics, 21, 1979.
- M. Raissi, P. Perdikaris, G. E. Karniadakis – Physics-informed neural networks, Journal of Computational Physics, 378, 2019.
- Z. Li et al. – Fourier Neural Operator for parametric partial differential equations, ICLR 2021.
- S. L. Brunton, J. L. Proctor, J. N. Kutz – Discovering governing equations from data by sparse identification of nonlinear dynamical systems, PNAS, 113(15), 2016.
- R. Lam et al. – Learning skillful medium-range global weather forecasting, Science, 382, 2023; I. Price et al. – Probabilistic weather forecasting with machine learning, Nature, 637, 2025 (online decembrie 2024); D. Kochkov et al. – Neural general circulation models for weather and climate, Nature, 632, 2024.
- ECMWF – AIFS Single (operațional din februarie 2025) și AIFS ENS (operațional din 1 iulie 2025), ecmwf.int.
- J. Tobin et al. – Domain randomization for transferring deep neural networks from simulation to the real world, IROS 2017; OpenAI et al. – Solving Rubik's Cube with a robot hand, arXiv:1910.07113, 2019.
- R. E. Kalman – A new approach to linear filtering and prediction problems, Journal of Basic Engineering, 82, 1960.
- National Academies of Sciences, Engineering, and Medicine – Foundational Research Gaps and Future Directions for Digital Twins, The National Academies Press, 2024.
- W. Kritzinger et al. – Digital Twin in manufacturing: a categorical literature review and classification, IFAC-PapersOnLine, 51(11), 2018; Destination Earth (destination-earth.eu).
- Material realizat cu ajutorul Claude Fable 5.1 și Claude Opus 5.5